

PRASAD V. POTLURI SIDDHARTHA INSTITUTE OF TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

PROJECT REPORT

PLACEMENTAI

RAG-BASED STUDENT PLACEMENT ASSISTANT

A Generative AI Application using Retrieval-Augmented Generation

Submitted by

Student Name: Mahammad Sathika

Roll Number: 23501A05A9

Branch: Computer Science and Engineering (CSE)

Section: 2

Year / Semester: 4-1

Project Type

Individual Generative AI Assignment

Submitted as part of the Generative AI Course Project

DECLARATION

I, **Mahammad Sathika**, bearing Roll Number **23501A05A9**, a student of the Department of Computer Science and Engineering at Prasad V. Potluri Siddhartha Institute of Technology, hereby declare that the project titled “**PlacementAI – RAG-Based Student Placement Assistant**” is my work carried out as part of the Generative AI course project.

This project focuses on the development of an intelligent student placement assistance application using **Retrieval-Augmented Generation (RAG)**. The system combines document retrieval, semantic embeddings, vector similarity search, and a Large Language Model to provide placement-related information to students.

The project has been developed for academic purposes and demonstrates the practical application of Generative AI, Natural Language Processing, document processing, semantic search, vector databases, and conversational AI.

I further declare that the information presented in this report accurately describes the design, implementation, technologies, methodology, and functionality of the developed system.

Student Name: Mahammad Sathika

Roll Number: 23501A05A9

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **Prasad V. Potluri Siddhartha Institute of Technology** and the **Department of Computer Science and Engineering** for providing the academic environment, guidance, and resources necessary to complete this project.

I am grateful for the opportunity provided through the Generative AI course to work on a practical application involving modern Artificial Intelligence technologies. This project helped me understand the practical implementation of **Retrieval-Augmented Generation, semantic embeddings, vector databases, Large Language Models, document processing, and conversational interfaces**.

I would also like to acknowledge the open-source frameworks and libraries used in the development of this project, including Python, Streamlit, LangChain, FAISS, Sentence Transformers, PyPDF, python-docx, and other supporting libraries.

Finally, I would like to thank everyone who provided technical guidance, suggestions, and encouragement during the development and documentation of this project.

ABSTRACT

PlacementAI is a Generative AI-powered student placement assistance system developed using **Retrieval-Augmented Generation (RAG)**. The primary purpose of the application is to provide students with quick and relevant information related to placement activities such as placement policies, eligibility criteria, registration procedures, placement rules, resume preparation, aptitude preparation, technical interviews, HR interviews, internships, offer letters, and frequently asked placement questions.

Traditional information-search methods require students to manually search through multiple documents to locate relevant information. This process can be time-consuming, especially when the required information is distributed across different documents. PlacementAI addresses this problem by creating a centralized knowledge base and allowing students to interact with it through a conversational interface.

The application first processes placement-related documents and divides their content into smaller searchable chunks. Each chunk is converted into a numerical vector representation using the **all-MiniLM-L6-v2** sentence embedding model. These embeddings are stored in a **FAISS vector database**, which enables semantic similarity search.

When a student enters a question, PlacementAI retrieves the most relevant document chunks from the vector database. The retrieved information is then provided as context to **Gemini 2.5 Flash**, which generates a natural-language answer based on the retrieved knowledge.

The system is implemented using **Python, Streamlit, LangChain, FAISS, Hugging Face Sentence Transformers, PyPDF, python-docx, and Google Gemini**. It supports TXT, Markdown, PDF, and DOCX knowledge documents. The application also allows users to upload additional documents and rebuild the RAG knowledge base dynamically.

An important feature of PlacementAI is source attribution. The application displays the source documents associated with retrieved information, helping users understand where the generated response originated.

The project demonstrates how Retrieval-Augmented Generation can be used to build a domain-specific educational assistant while reducing unsupported or fabricated answers from a general-purpose language model.

Keywords: Generative AI, Retrieval-Augmented Generation, RAG, Large Language Model, Gemini, FAISS, Embeddings, Semantic Search, LangChain, Streamlit, Student Placement Assistant.

TABLE OF CONTENTS

1. Introduction
2. Problem Statement
3. Motivation
4. Objectives
5. Scope of the Project

6. Proposed System
 7. Existing System
 8. Limitations of Existing System
 9. System Architecture
 10. Knowledge Base
 11. Retrieval-Augmented Generation Methodology
 12. Document Processing and Chunking
 13. Embedding Generation
 14. FAISS Vector Database
 15. Large Language Model Integration
 16. Prompt Design
 17. User Interface
 18. Document Upload System
 19. Conversation and Source Attribution
 20. Technology Stack
 21. Project Structure
 22. Detailed System Workflow
 23. Implementation Details
 24. Functional Modules
 25. Testing
 26. Results and Discussion
 27. Advantages
 28. Limitations
 29. Security Considerations
 30. Future Enhancements
 31. Conclusion
 32. References
 33. Appendix
-

1. INTRODUCTION

Artificial Intelligence has become an important technology for developing intelligent educational and student-support applications. With the development of Large Language Models, users can interact with computer systems using natural language instead of traditional search interfaces.

However, general-purpose Large Language Models may not always contain the latest or institution-specific information required by students. Placement-related information is often distributed across multiple documents such as placement policies, eligibility rules, registration instructions, interview guidelines, internship information, and offer-letter procedures.

Students may therefore spend significant time searching through different documents to locate the required information.

To address this problem, **PlacementAI – RAG-Based Student Placement Assistant** has been developed as a domain-specific Generative AI application.

PlacementAI uses the Retrieval-Augmented Generation architecture. Instead of asking the Large Language Model to answer every question solely from its internal knowledge, the application first searches a placement-specific knowledge base and retrieves relevant information. The retrieved information is then supplied to the language model as context for generating the final response.

The application is implemented using Python and Streamlit. LangChain is used to connect the retrieval and generation components, FAISS is used for vector similarity search, the all-MiniLM-L6-v2 model is used for semantic embeddings, and Gemini 2.5 Flash is used as the Large Language Model.

The application supports TXT, Markdown, PDF, and DOCX documents and provides functionality for adding new documents to the knowledge base.

2. PROBLEM STATEMENT

Students frequently require information regarding campus placements, but such information may be distributed across multiple documents and sources.

Typical student queries include:

- What are the eligibility requirements for placements?
- How do I register for placements?
- What are the placement rules?
- How should I prepare my resume?
- How can I prepare for aptitude tests?
- What topics should I study for technical interviews?
- How should I prepare for HR interviews?
- What are the internship guidelines?
- What information is included in an offer letter?
- What are the frequently asked placement questions?

Searching manually through several documents can be inefficient.

The problem addressed by this project is therefore:

“How can Generative AI and Retrieval-Augmented Generation be used to create an intelligent student placement assistant that retrieves relevant

placement information from a domain-specific knowledge base and provides accurate natural-language responses?”

3. MOTIVATION

The main motivation for developing PlacementAI is to simplify access to placement-related information.

Students generally interact with information through websites, notices, PDF documents, circulars, and other institutional material. A conversational AI interface can make this information easier to access.

The project is also motivated by the need to understand the practical implementation of modern Generative AI systems.

Through this project, the following concepts were explored:

- Large Language Models
 - Retrieval-Augmented Generation
 - Natural Language Processing
 - Semantic Search
 - Text Embeddings
 - Vector Databases
 - Document Processing
 - Prompt Engineering
 - Conversational AI
 - Generative AI application development
-

4. OBJECTIVES

The major objectives of PlacementAI are:

1. To create a centralized knowledge base containing placement-related documents.
2. To process different document formats into machine-readable text.
3. To divide large documents into smaller searchable chunks.
4. To generate semantic embeddings for document chunks.
5. To store document embeddings in a vector database.
6. To perform semantic similarity search for student queries.
7. To retrieve relevant information from the placement knowledge base.
8. To provide retrieved information to a Large Language Model.

9. To generate natural-language responses using Gemini 2.5 Flash.
 10. To reduce unsupported answers by instructing the model to rely on retrieved knowledge.
 11. To display source information associated with generated answers.
 12. To provide a simple conversational user interface.
 13. To support uploading additional placement-related documents.
 14. To dynamically rebuild the RAG system after adding new documents.
 15. To demonstrate a practical application of Generative AI in education and student placement assistance.
-

5. SCOPE OF THE PROJECT

The scope of PlacementAI is focused primarily on student placement-related information.

The system can be used for:

- Placement policy information
- Eligibility information
- Placement registration
- Placement rules
- Resume preparation
- Aptitude preparation
- Technical interview preparation
- HR interview preparation
- Group discussion preparation
- Internship information
- Offer-letter information
- Frequently asked placement questions

The system can also be extended by uploading additional documents in supported formats.

The current implementation supports:

- TXT files
- Markdown files
- PDF files
- DOCX files

The system is designed as a domain-specific assistant rather than a general-purpose chatbot.

6. EXISTING SYSTEM

In a conventional placement information system, students typically obtain information through:

- College websites
- Placement cell notices
- PDF documents
- Circulars
- Emails
- Student coordinators
- Faculty members
- Manual document searching

These methods can work effectively when the number of documents is small. However, as the amount of information increases, finding specific information becomes more difficult.

Traditional keyword-based search can also fail when the user and document use different words for the same concept.

For example, a student might ask:

“Can I participate in campus recruitment?”

while the document may contain:

“Eligibility criteria for placement registration.”

A semantic retrieval system can recognize the relationship between these concepts better than a simple keyword search.

7. LIMITATIONS OF EXISTING SYSTEM

The conventional approach has several limitations:

1. Information is distributed across multiple sources.
2. Students must manually search through documents.
3. Keyword-based search may not understand semantic meaning.
4. Students may have difficulty locating the exact information required.
5. Repeated questions may need to be answered manually by placement coordinators.

6. Students may depend on outdated documents.
7. Searching multiple documents can consume significant time.
8. There is no single conversational interface for asking placement-related questions.

PlacementAI attempts to address these limitations through semantic retrieval and Generative AI.

8. PROPOSED SYSTEM

The proposed system is **PlacementAI**, a Retrieval-Augmented Generation based student placement assistant.

The system contains the following major stages:

1. Knowledge base creation
2. Document loading
3. Text extraction
4. Document chunking
5. Embedding generation
6. FAISS vector database creation
7. User query processing
8. Similarity-based retrieval
9. Context construction
10. LLM response generation
11. Source display

The user interacts with the system through a Streamlit web interface.

The user enters a placement-related question. The system searches the vector database for the four most relevant document chunks. These chunks are combined into a context and provided to Gemini 2.5 Flash.

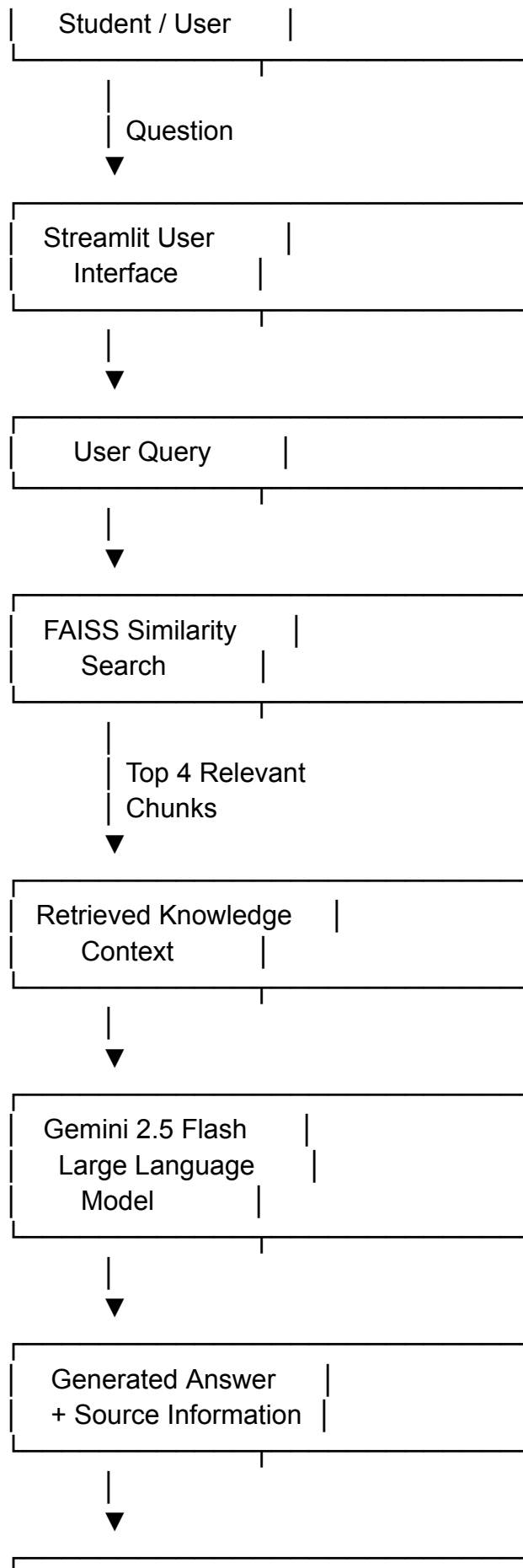
The LLM is instructed to answer using only the retrieved placement knowledge base.

This design reduces the possibility of unsupported placement-specific claims.

9. SYSTEM ARCHITECTURE

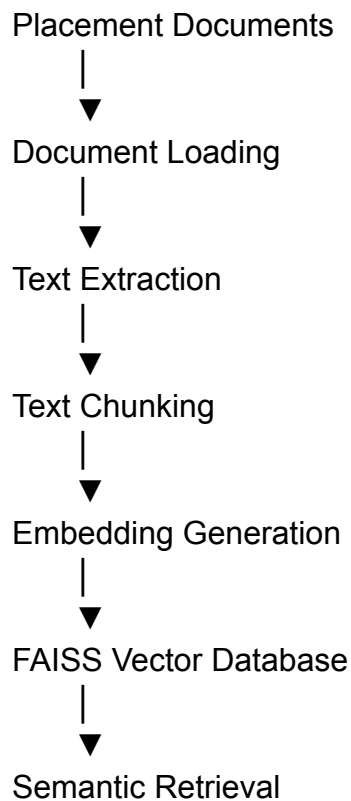
The architecture of PlacementAI can be represented as follows:







The knowledge-base creation pipeline works as follows:



10. KNOWLEDGE BASE

The PlacementAI knowledge base is designed around placement-related information.

The major document categories include:

No	Document / Topic
----	------------------

- | | |
|---|----------------------|
| . | |
| 1 | Placement Policy |
| 2 | Eligibility Criteria |
| 3 | Registration Process |
| 4 | Placement Rules |

- 5 Resume Guidelines
- 6 Aptitude Preparation
- 7 Technical Interview
- 8 HR Interview
- 9 Group Discussion
- 10 Internship Guidelines
- 11 Offer Letter
Guidelines
- 12 Placement FAQ

These documents provide the domain-specific information used by the RAG system.

The knowledge base is stored in a dedicated `knowledge_base` directory. The application automatically searches this directory for supported document formats.

The implementation supports `.txt`, `.md`, `.pdf`, and `.docx` files.

11. RETRIEVAL-AUGMENTED GENERATION METHODOLOGY

Retrieval-Augmented Generation combines two major operations:

1. Information retrieval
2. Language generation

A traditional LLM generates an answer based primarily on its learned parameters.

A RAG system first searches an external knowledge source and then uses the retrieved information to generate the response.

PlacementAI follows the following RAG process:

User Question



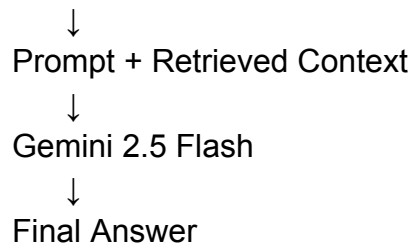
Semantic Similarity Search



Relevant Document Chunks



Context Construction



The implementation retrieves **four relevant documents/chunks** using FAISS similarity search.

The retrieved information is then combined into a context before being supplied to Gemini.

12. DOCUMENT PROCESSING AND CHUNKING

Large documents cannot always be directly passed to a retrieval system as a single unit.

Therefore, PlacementAI divides documents into smaller chunks.

The project uses LangChain's `RecursiveCharacterTextSplitter`.

The configured parameters are:

- Chunk size: **800 characters**
- Chunk overlap: **150 characters**

The chunk splitter uses separators including:

- Paragraph breaks
- Line breaks
- Sentence boundaries
- Spaces
- Individual characters

This approach attempts to preserve meaningful sections of text while creating manageable chunks for semantic retrieval.

Why Chunking is Important

Chunking improves retrieval because:

- Smaller sections are easier to search.

- Relevant information can be retrieved without processing an entire document.
- Semantic embeddings can represent specific portions of the document.
- Retrieved context can remain focused on the user's question.

The overlap of 150 characters helps preserve contextual information between neighboring chunks.

13. DOCUMENT LOADING

PlacementAI supports four major document types:

13.1 TXT Files

Plain-text files are read using UTF-8 encoding.

13.2 Markdown Files

Markdown documents are processed similarly to text files.

13.3 PDF Files

PDF files are processed using the [pypdf](#) library. Text is extracted page by page, and the page number is stored as metadata.

13.4 DOCX Files

DOCX files are processed using [python-docx](#). Paragraph text is extracted and combined into a document representation.

The source filename is stored as metadata so that retrieved information can later be associated with its source. PDF documents additionally store their page number.

14. EMBEDDING GENERATION

Text embeddings convert textual information into numerical vector representations.

PlacementAI uses the:

all-MiniLM-L6-v2

embedding model from the Sentence Transformers ecosystem.

The model converts each document chunk into a vector representation that captures semantic information.

For example, the following two questions may have similar meanings:

- “What is the eligibility for placements?”
- “Who can participate in campus recruitment?”

Although the exact words differ, their semantic meanings are related.

Embeddings help the retrieval system identify such relationships.

The application creates embeddings using [HuggingFaceEmbeddings](#) with the [sentence-transformers/all-MiniLM-L6-v2](#) model.

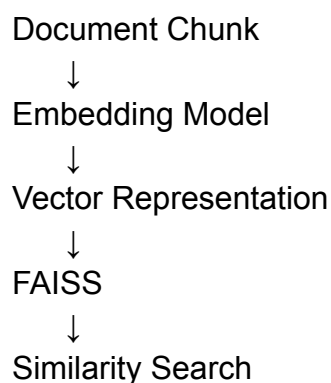
15. FAISS VECTOR DATABASE

FAISS stands for **Facebook AI Similarity Search**.

It is a library designed for efficient similarity search over vector representations.

PlacementAI uses FAISS to store the embeddings generated from document chunks.

The process is:



When a user enters a question, the system compares the query against the stored vectors and retrieves the most semantically relevant chunks.

The application creates the FAISS database directly from the processed document chunks and embeddings.

16. LARGE LANGUAGE MODEL INTEGRATION

PlacementAI uses **Gemini 2.5 Flash** as its Large Language Model.

The Gemini model is accessed through LangChain's Google Generative AI integration.

The configured model is:

gemini-2.5-flash

The application uses a temperature value of **0.2**.

A relatively low temperature is suitable for a knowledge-based assistant because the system should prefer consistent and grounded responses rather than highly creative responses.

The API key is loaded from an environment variable instead of being hard-coded into the Python application.

17. PROMPT DESIGN

Prompt design is an important part of the RAG system.

PlacementAI instructs Gemini to behave as a student placement assistant.

The prompt contains several important rules.

The model is instructed to:

1. Use the retrieved context as its primary source.
2. Avoid inventing placement policies.
3. Avoid inventing eligibility rules.
4. Avoid inventing dates.
5. Avoid inventing procedures.
6. Avoid inventing company requirements.
7. State that relevant information could not be found when the knowledge base does not contain the answer.
8. Explain politely when the question is unrelated to student placements.
9. Provide a clear and useful response.

The prompt therefore acts as a grounding mechanism between the retrieval system and the LLM.

18. USER INTERFACE

The application uses **Streamlit** to provide a web-based conversational interface.

The main interface contains:

- Application title
- Description
- Chat input
- Conversation history
- Assistant responses
- Source information
- Knowledge-base statistics
- Document upload functionality

The main page is titled:

PlacementAI – RAG-Based Student Placement Assistant

Users can directly enter placement-related questions using the chat input.

The application maintains messages using Streamlit session state and displays previous user and assistant messages during the session.

19. SIDEBAR INFORMATION

The sidebar provides information about the current RAG system.

It displays:

- Number of knowledge documents
- Number of text chunks
- Embedding model
- Vector database
- Large Language Model

The application displays:

Embedding Model: all-MiniLM-L6-v2

Vector Database: FAISS

LLM: Gemini 2.5 Flash

The sidebar also explains that the assistant retrieves relevant information from the placement knowledge base before generating an answer.

20. DOCUMENT UPLOAD SYSTEM

One of the important features of PlacementAI is the ability to add new documents to the knowledge base.

The application provides an upload interface in the sidebar.

Supported formats are:

- TXT
- Markdown
- PDF
- DOCX

Multiple documents can be uploaded at the same time.

After selecting the files, the user can click:

“Add to Knowledge Base”

The application saves supported documents into the knowledge base directory and rebuilds the RAG components.

The upload functionality is implemented through Streamlit's file uploader and the application subsequently clears the cached document, chunk, and vector-database resources so that the new documents can be included.

21. KNOWLEDGE BASE FILE MANAGEMENT

The application also displays the files currently available in the knowledge base.

This allows the user to see which placement documents are being used by the system.

The system checks file extensions and displays supported knowledge-base files in the sidebar.

This provides basic transparency regarding the documents currently used for retrieval.

22. CONVERSATION AND SOURCE ATTRIBUTION

PlacementAI provides a conversational interaction model.

When a user asks a question:

1. The question is displayed in the chat.
2. The RAG retrieval process is executed.
3. The relevant documents are retrieved.
4. Gemini generates an answer.
5. The answer is displayed.
6. Source filenames are displayed below the response.

For PDF documents, the page number can also be included in the displayed source.

The implementation stores source names in the assistant message and displays them using a “Sources” label.

This feature improves transparency because the user can identify the document from which the retrieved information originated.

23. TECHNOLOGY STACK

Technology	Purpose
Python	Core programming language
Streamlit	Web application and user interface
LangChain	RAG application framework
Gemini 2.5 Flash	Large Language Model
FAISS	Vector similarity search
all-MiniLM-L6-v2	Text embedding model
Sentence Transformers	Embedding generation
PyPDF	PDF text extraction
python-docx	DOCX document processing

python-dotenv	Environment configuration
NumPy	Numerical computation
scikit-learn	Supporting machine-learning functionality
Hugging Face	Embedding model ecosystem

The actual project dependencies include Streamlit, FAISS CPU, LangChain, LangChain Google GenAI, LangChain HuggingFace, LangChain text splitters, pypdf, python-docx, python-dotenv, sentence-transformers, transformers, and related packages.

24. PROJECT STRUCTURE

The project can be organized in the following structure:

```
PlacementAI/
|
|— app.py
|
|— requirements.txt
|
|— .env
|
|— knowledge_base/
    |
    |— placement_policy
    |— eligibility_criteria
    |— registration_process
    |— placement_rules
    |— resume_guidelines
    |— aptitude_preparation
    |— technical_interview
    |— hr_interview
    |— group_discussion
    |— internship_guidelines
    |— offer_letter_guidelines
    |— placement_faq
```

The main application file is `app.py`.

The knowledge base directory is created automatically by the application if it does not already exist.

The `requirements.txt` file contains the Python dependencies necessary to run the project.

The `.env` file is used to provide the Google API key through the `GOOGLE_API_KEY` environment variable. The actual secret value must not be included in the submitted report or shared publicly.

25. DETAILED SYSTEM WORKFLOW

The complete workflow of PlacementAI is divided into two phases.

Phase 1: Knowledge Base Creation

Step 1: Collect Documents

Placement-related documents are collected and stored in the knowledge base directory.

Step 2: Load Documents

The application identifies supported files.

Step 3: Extract Text

Text is extracted from TXT, Markdown, PDF, and DOCX files.

Step 4: Create Chunks

Documents are divided into smaller chunks using recursive character-based splitting.

Step 5: Generate Embeddings

Each chunk is converted into an embedding using all-MiniLM-L6-v2.

Step 6: Store Embeddings

The embeddings are stored in a FAISS vector database.

Phase 2: Question Answering

Step 1: User Enters Question

The student enters a placement-related question.

Step 2: Similarity Search

FAISS searches the knowledge base for relevant chunks.

Step 3: Retrieve Top Results

The application retrieves four relevant document chunks.

Step 4: Build Context

The retrieved chunks are combined into a context.

Step 5: Generate Prompt

The context and user question are inserted into the RAG prompt.

Step 6: Call Gemini

Gemini 2.5 Flash generates the response.

Step 7: Display Answer

The response is shown in the Streamlit interface.

Step 8: Display Sources

The associated source documents are shown below the response.

26. IMPLEMENTATION DETAILS

26.1 Environment Configuration

The application uses the `python-dotenv` library to load environment variables.

The Google API key is accessed through:

```
GOOGLE_API_KEY
```

The application checks whether the key exists before starting the RAG system. If the key is missing, the application displays an error and stops execution.

26.2 Document Loading

The application uses a dedicated document-reading function.

The function determines the file type from its extension and applies the corresponding extraction method.

This creates LangChain **Document** objects containing:

- Page content
- Source filename
- PDF page number when applicable

26.3 Chunk Creation

The document chunks are created using:

```
chunk_size = 800  
chunk_overlap = 150
```

This process allows the vector database to perform more focused retrieval.

26.4 Vector Database Creation

The embedding model is initialized and the document chunks are passed to FAISS.

The resulting vector database can perform similarity searches based on user questions.

26.5 Query Processing

When a question is submitted, the application executes:

```
similarity_search(question, k=4)
```

Therefore, the system attempts to retrieve the four most relevant document chunks.

26.6 Context Construction

For every retrieved document, the system obtains:

- Source name
- Page number when available
- Document content

These are combined to create the retrieved context.

26.7 Response Generation

The context and user question are passed to the Gemini model through a LangChain prompt chain.

The final response is returned to the Streamlit interface.

27. FUNCTIONAL MODULES

PlacementAI can be divided into the following functional modules.

Module 1: Environment Configuration

Responsible for loading the API key and application configuration.

Module 2: Document Loader

Responsible for reading TXT, Markdown, PDF, and DOCX files.

Module 3: Text Chunking

Responsible for dividing documents into searchable chunks.

Module 4: Embedding Generator

Responsible for converting text chunks into vector representations.

Module 5: FAISS Retriever

Responsible for semantic similarity search.

Module 6: Gemini LLM

Responsible for natural-language response generation.

Module 7: Prompt System

Responsible for grounding the LLM response in retrieved context.

Module 8: Chat Interface

Responsible for user interaction and displaying answers.

Module 9: Document Upload

Responsible for adding new documents.

Module 10: Source Attribution

Responsible for displaying the source documents used for the response.

28. TESTING

Testing was performed conceptually using different categories of placement-related questions.

Test Case	Input	Expected Result
TC01	Eligibility question	Relevant eligibility information
TC02	Registration question	Registration procedure
TC03	Resume question	Resume guidance
TC04	Aptitude question	Aptitude preparation information
TC05	Technical interview question	Technical preparation guidance
TC06	HR interview question	HR interview guidance
TC07	Internship question	Internship guidelines

TC08	Offer-letter question	Offer-letter information
TC09	Uploaded document question	Information from uploaded content
TC10	Unrelated question	System should avoid unrelated knowledge-base claims

29. TEST CASE DETAILS

TC01 – Eligibility Query

Input:

“What are the eligibility requirements for placement?”

Expected Result:

The system retrieves relevant eligibility information and provides an answer based on the placement knowledge base.

TC02 – Registration Query

Input:

“How can I register for placements?”

Expected Result:

The system retrieves information related to the registration process.

TC03 – Resume Query

Input:

“How should I prepare my resume?”

Expected Result:

The system retrieves resume-related information and provides guidance.

TC04 – Aptitude Query

Input:

“How can I prepare for the aptitude test?”

Expected Result:

The system retrieves relevant aptitude preparation material.

TC05 – Technical Interview Query**Input:**

“What should I prepare for a technical interview?”

Expected Result:

The system retrieves technical interview-related content.

TC06 – HR Interview Query**Input:**

“How should I prepare for an HR interview?”

Expected Result:

The system retrieves relevant HR interview information.

TC07 – Internship Query**Input:**

“What are the internship guidelines?”

Expected Result:

The system retrieves internship-related information.

TC08 – Offer Letter Query**Input:**

“What should I know about an offer letter?”

Expected Result:

The system retrieves offer-letter information.

TC09 – Uploaded Document Query

Input:

A question related to newly uploaded placement material.

Expected Result:

The uploaded document becomes part of the knowledge base after rebuilding the RAG system, and relevant information can subsequently be retrieved.

TC10 – Unsupported Query

Input:

“Tell me about a topic unrelated to student placements.”

Expected Result:

The system should explain that it is designed primarily for placement-related questions instead of presenting unsupported placement information.

30. RESULTS AND DISCUSSION

The developed PlacementAI system provides a functional RAG-based conversational interface for student placement assistance.

The major functional outcomes of the project are:

30.1 Domain-Specific Question Answering

The application can retrieve information from a placement-specific knowledge base and generate natural-language responses.

30.2 Semantic Retrieval

The system uses embeddings and FAISS similarity search rather than relying only on exact keyword matching.

30.3 Context-Aware Generation

The retrieved document chunks are passed to Gemini as context before generating an answer.

30.4 Source Display

The system displays source filenames associated with retrieved information.

30.5 Multiple Document Formats

The application can process TXT, Markdown, PDF, and DOCX files.

30.6 Dynamic Knowledge Base Expansion

Users can upload additional documents through the Streamlit interface.

30.7 Conversational Interface

Students can interact with the system using a chat-style interface.

30.8 Grounded Responses

The prompt instructs Gemini to avoid inventing placement-specific information and to state when relevant information cannot be found.

Overall, the project demonstrates the practical integration of a vector database, embedding model, document processing pipeline, and Large Language Model into a Generative AI application.

31. ADVANTAGES

The major advantages of PlacementAI are:

1. **Faster access to placement information**
Students can ask questions directly instead of manually searching multiple documents.
2. **Centralized knowledge base**
Placement-related information can be maintained in one location.
3. **Semantic search**
The system can retrieve conceptually relevant information even when the exact words differ.
4. **Natural-language interaction**
Students can ask questions using normal language.
5. **LLM-powered responses**
Retrieved information is converted into readable natural-language answers.
6. **Source attribution**
Source information can be displayed with generated responses.

7. Multiple document formats

TXT, Markdown, PDF, and DOCX documents are supported.

8. Expandable knowledge base

New documents can be uploaded without rewriting the main application.

9. Domain-focused responses

The prompt restricts the model to retrieved placement knowledge.

10. Simple user interface

Streamlit provides a straightforward conversational interface.

32. LIMITATIONS

Despite its advantages, the system has several limitations.

32.1 Knowledge Base Dependency

The quality of responses depends on the quality and completeness of the documents available in the knowledge base.

32.2 Outdated Information

If placement documents are outdated, the system may retrieve outdated information.

32.3 API Dependency

Gemini interaction requires a valid Google API key and network connectivity.

32.4 Ambiguous Queries

Questions that are unclear or overly broad may not retrieve the most appropriate information.

32.5 Domain Limitation

The application is primarily designed for student placement-related questions.

32.6 Retrieval Dependency

If the relevant information is not retrieved, the LLM may not have enough context to answer the question.

32.7 Local Knowledge Base

The current implementation relies on the documents available to the application rather than a centralized institutional database.

33. SECURITY CONSIDERATIONS

Security is an important aspect of a Generative AI application.

The following security considerations apply to PlacementAI:

33.1 API Key Protection

The Google API key should be stored in an environment file and should never be hard-coded into source code.

The `.env` file should not be uploaded to public repositories.

33.2 Sensitive Information

Users should avoid uploading confidential student information, personal identification documents, passwords, or other sensitive data unless appropriate security controls are implemented.

33.3 Document Validation

The application accepts only supported document extensions:

- TXT
- Markdown
- PDF
- DOCX

33.4 Knowledge Base Security

Uploaded documents should be trusted and verified before being used as official placement information.

33.5 Prompt Security

The system prompt instructs the model to use retrieved knowledge and avoid inventing placement-specific policies.

33.6 Secret Management

Environment variables should be used for credentials and API keys.

Important deployment practice: the `.env` file containing the actual API key should never be included in a GitHub repository, project report, screenshots, or public submission.

34. FUTURE ENHANCEMENTS

PlacementAI can be extended with several advanced features.

34.1 Personalized Eligibility Checking

The system could collect student information and automatically determine whether the student satisfies the eligibility criteria for a particular company.

34.2 Resume Upload and Analysis

Students could upload their resumes and receive AI-generated suggestions for improvement.

34.3 Resume Scoring

The application could provide a score based on resume structure, skills, projects, education, and job requirements.

34.4 Mock Interviews

The system could conduct simulated technical and HR interviews.

34.5 Company-Specific Preparation

Students could select a company and receive company-specific preparation material.

34.6 Aptitude Practice

The system could generate aptitude questions and track student performance.

34.7 Performance Analytics

The application could maintain statistics about student preparation and improvement.

34.8 Authentication

User authentication could be introduced to provide personalized dashboards.

34.9 Administrator Controls

Placement administrators could manage official placement documents.

34.10 Cloud Deployment

The application could be deployed on a cloud platform for wider student access.

34.11 Database Integration

A structured database could store student profiles, placement information, interview records, and analytics.

34.12 Multilingual Support

The assistant could be extended to support multiple Indian languages.

34.13 Improved Retrieval

Hybrid retrieval combining keyword search and vector search could improve retrieval quality.

34.14 Reranking

A reranking model could be added to improve the ordering of retrieved documents.

35. CONCLUSION

PlacementAI demonstrates the practical application of Generative AI and Retrieval-Augmented Generation in the field of student placement assistance.

The system creates a domain-specific knowledge base from placement-related documents, processes those documents into searchable chunks, converts them into semantic embeddings, and stores the embeddings in a FAISS vector database.

When a student asks a question, the system retrieves relevant information using semantic similarity search. The retrieved context is then passed to Gemini 2.5 Flash, which generates a natural-language response.

The application also provides source attribution, document uploading, multiple document-format support, and a conversational Streamlit interface.

The project successfully demonstrates the integration of several modern AI technologies into a single application:

- Generative AI
- Retrieval-Augmented Generation
- Large Language Models
- Semantic Embeddings
- Vector Databases
- Natural Language Processing
- Document Processing
- Conversational AI

The project provides a foundation for developing more advanced student-support systems in the future. Features such as personalized eligibility checking, resume analysis, mock interviews, company-specific preparation, authentication, analytics, and cloud deployment can further improve the system.

Overall, PlacementAI demonstrates how Retrieval-Augmented Generation can be used to create a practical, domain-specific Generative AI application that helps students access placement information more efficiently.

36. REFERENCES

1. Python Documentation – Python programming language and standard library documentation.
2. Streamlit Documentation – Documentation for developing interactive Python-based web applications.
3. LangChain Documentation – Framework documentation for developing applications powered by Large Language Models.
4. FAISS Documentation – Documentation and research resources related to efficient similarity search and vector indexing.
5. Sentence Transformers Documentation – Documentation for semantic text embeddings.
6. Google Gemini API Documentation – Documentation for Gemini Large Language Models and API integration.
7. PyPDF Documentation – Documentation for PDF document processing and text extraction.
8. python-docx Documentation – Documentation for processing Microsoft Word DOCX documents.
9. Hugging Face Documentation – Resources related to transformer models and machine-learning models.

10. Educational resources related to Retrieval-Augmented Generation, semantic search, vector embeddings, prompt engineering, and Large Language Models.

37. APPENDIX A – PROJECT INFORMATION

Parameter	Details
Project Name	PlacementAI – RAG-Based Student Placement Assistant
Student Name	Mahammad Sadhika
Roll Number	23501A05A9
Branch	Computer Science and Engineering
Section	2
Year / Semester	4-1
Institution	Prasad V. Potluri Siddhartha Institute of Technology
Domain	Generative AI / Education / Student Placement
Architecture	Retrieval-Augmented Generation
LLM	Gemini 2.5 Flash
Embedding Model	all-MiniLM-L6-v2
Vector Database	FAISS
Application Framework	Streamlit
RAG Framework	LangChain
Supported Documents	TXT, Markdown, PDF, DOCX
Retrieval Count	Top 4 relevant chunks
Chunk Size	800 characters
Chunk Overlap	150 characters

Core Features	Semantic retrieval, LLM response generation, source display, document upload, conversational interface
---------------	--

38. APPENDIX B – IMPORTANT CODE COMPONENTS

Application Initialization

The application configures Streamlit and initializes the environment required by the RAG system.

Knowledge Base Directory

The application creates and uses a dedicated knowledge base directory for placement documents.

Supported File Types

.txt
.md
.pdf
.docx

Embedding Model

sentence-transformers/all-MiniLM-L6-v2

Vector Database

FAISS

Language Model

gemini-2.5-flash

Chunk Configuration

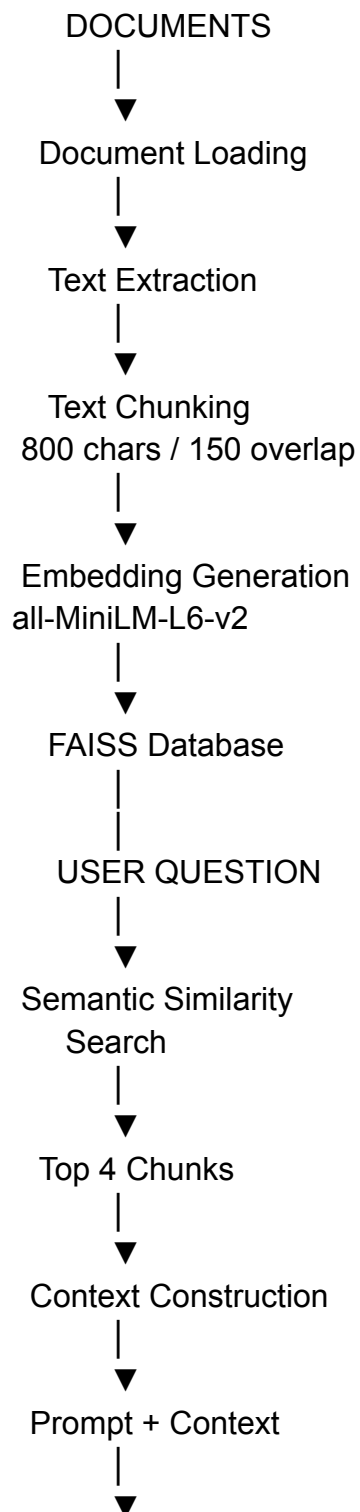
Chunk Size: 800
Chunk Overlap: 150

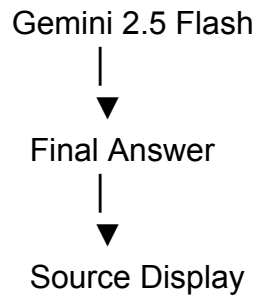
Retrieval Configuration

Top K: 4

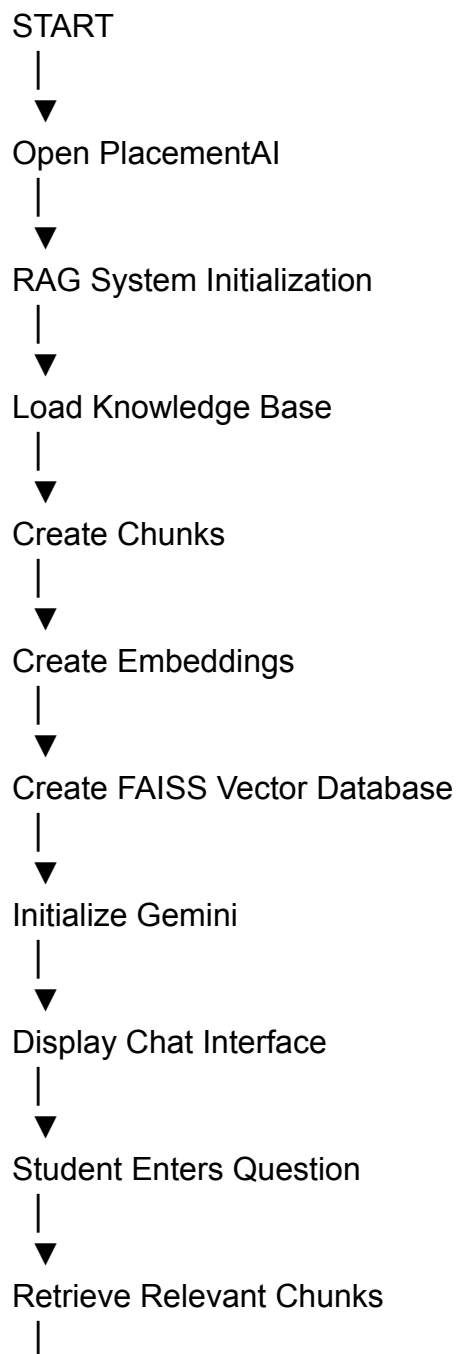
39. APPENDIX C – RAG PIPELINE

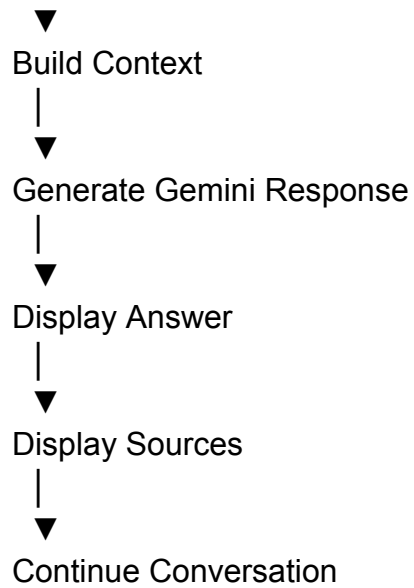
The complete RAG pipeline can be summarized as:





40. APPENDIX D – USER INTERACTION FLOW





41. APPENDIX E – SAMPLE QUESTIONS

The following questions can be used to demonstrate the application:

Placement Eligibility

“What are the eligibility requirements for placement?”

Registration

“How do I register for campus placements?”

Resume

“What should I include in my placement resume?”

Aptitude

“How can I prepare for placement aptitude tests?”

Technical Interview

“What topics should I prepare for a technical interview?”

HR Interview

“What questions are commonly asked in HR interviews?”

Group Discussion

“How can I perform better in a group discussion?”

Internship

“What are the internship guidelines?”

Offer Letter

“What should I check in an offer letter?”

Placement Rules

“What are the important placement rules?”

42. APPENDIX F – SAMPLE OUTPUT FORMAT

A typical interaction with the system follows this structure:

USER:

What are the eligibility requirements for placement?

PLACEMENTAI:

[Generated answer based on the retrieved placement knowledge-base information.]

SOURCES:

placement_policy.pdf
eligibility_criteria.pdf

The exact response depends on the documents available in the knowledge base and the question asked by the user.

43. APPENDIX G – SOFTWARE REQUIREMENTS

The project requires:

- Python
- Streamlit
- LangChain
- FAISS
- Sentence Transformers
- Hugging Face Transformers
- Google Gemini integration
- PyPDF
- python-docx
- python-dotenv

The project's dependency file includes the required libraries for the application, including Streamlit, FAISS CPU, LangChain components, Gemini integration, Hugging Face integration, PDF processing, DOCX processing, Sentence Transformers, and supporting packages.

44. APPENDIX H – HARDWARE REQUIREMENTS

The application can be developed and executed on a standard modern computer.

Recommended minimum configuration:

- Processor: Intel Core i3 / equivalent or above
- RAM: 8 GB or above
- Storage: At least 5 GB available
- Operating System: Windows / Linux / macOS
- Internet connection: Required for Gemini API interaction

Additional resources may be required depending on the size of the knowledge base and embedding model execution.

45. APPENDIX I – KEY LEARNING OUTCOMES

Through the development of PlacementAI, the following learning outcomes were achieved:

1. Understanding of Generative AI applications.
2. Understanding of Retrieval-Augmented Generation.
3. Practical implementation of semantic search.
4. Understanding of text embeddings.
5. Practical use of FAISS vector databases.

6. Integration of Gemini Large Language Models.
 7. Document processing using Python.
 8. Development of conversational interfaces using Streamlit.
 9. Use of LangChain for RAG pipelines.
 10. Understanding of prompt grounding.
 11. Implementation of document upload functionality.
 12. Understanding of source attribution in AI applications.
 13. Understanding of environment-variable based API-key management.
 14. Practical experience in integrating multiple AI technologies into one application.
-

46. FINAL PROJECT SUMMARY

PlacementAI – RAG-Based Student Placement Assistant is a Generative AI application designed to simplify student access to placement-related information.

The project uses a Retrieval-Augmented Generation architecture consisting of:

Documents → Chunking → Embeddings → FAISS → Retrieval → Context → Gemini → Response

The combination of semantic retrieval and Large Language Model generation allows the system to provide natural-language answers while grounding the response in the available placement knowledge base.

The project demonstrates the practical use of modern Generative AI technologies for solving an educational information-access problem and provides a foundation for future development into a comprehensive AI-powered placement assistant.